

HURC — Production Build Prompt for Claude Code

Paste this entire document into Claude Code from an **empty folder**. Treat it as a senior engineer's technical specification. Do not skip phases. Do not stub. Do not invent versions — verify the latest stable versions of every package the moment you install it (`(pnpm view <pkg> version)`) and pin to exact minors.

0. Role & Operating Rules

You are acting as the **lead full-stack engineer** building HURC's production e-commerce platform from scratch. You own architecture, code, infra, tests, docs, and CI. You will not ask the user follow-up questions about scope — this document IS the scope. If you encounter a real ambiguity, decide using the principles in §15 ("Decision principles"), document the decision in `(/docs/decisions/NNNN-title.md)` (ADR format), and continue.

Hard rules — non-negotiable:

1. **Verify versions before installing.** Run `(pnpm view <pkg> version)` (or check the package's GitHub releases) before pinning. Do not trust this document's version hints blindly — they are minimums.
2. **TypeScript strict mode everywhere.** `("strict": true, "noUncheckedIndexedAccess": true, "exactOptionalPropertyTypes": true)`. No `(any)`. No `(// @ts-ignore)` without an attached justification comment.
3. **No shortcuts, no stubs, no TODOs left behind.** If a feature is in scope, it ships working. If you must defer, open a tracked issue in `(/docs/backlog.md)` with reasoning.
4. **Run quality gates after every phase.** `(pnpm typecheck && pnpm lint && pnpm test && pnpm build)` must pass before you mark a phase done.
5. **Accessibility is a PO acceptance criterion**, not a polish item. Every interactive component must be keyboard-navigable, screen-reader-labeled, and pass `(axe)` checks in tests.
6. **Production-grade error handling.** No swallowed exceptions, no `(console.log)` in shipped code (use the logger), no unhandled promise rejections. Every Server Action returns a discriminated union `({ ok: true, data } | { ok: false, error })`.
7. **Security defaults on.** CSP headers, HSTS, secure cookies, rate-limited mutations, validated inputs (Zod) at every trust boundary.
8. **Commit in small, conventional commits** `(feat:, fix:, chore:, docs:, etc.)` with imperative subjects. Each phase ends in a tagged commit `(phase-N-complete)`.
9. **If you don't know, look it up.** Read the actual Vendure docs, Next.js docs, Mollie docs. Do not hallucinate APIs.
10. **Idempotent scripts.** Every script in `(/scripts)` and `(package.json)` can be re-run safely.

1. Project Overview

Brand: HURC **Tagline:** Hustle Unleashed, Resilience Crafted **Backronym:** H.U.R.C. — Hustle Unleashed, Resilience Crafted **Category:** Premium activewear / sportswear DTC (Run / Train / Lift / Rest) **Markets:** EU primary (DE, FR, NL, BE, ES, IT) + UK. Currency: EUR primary, GBP for UK channel. Languages: en (default), de, fr, nl, es, it.

Brand identity:

- Primary: Black (■ #0A0A0A)
- Surface: White (□ #FFFFFF)
- Accent: Red (■ #E63946) — used as a *scarce* resource (CTAs, badges, sale ticks). Never as a fill on large surfaces.
- Aesthetic: Nordic editorial discipline (Peak Performance, Tiger of Sweden, Castore) × athletic energy (Gymshark, On Running). **Dark-default**, bold typography, generous whitespace, cinematic video hero, motion-conscious.
- Type: a high-contrast geometric display + a clean grotesque body. Use (next/font) with (Inter) (body) + (Bricolage Grotesque) or (Geist) (display) as a reasonable starting pair. Self-host, (font-display: swap).

2. Locked Tech Stack

This stack is **decided**. Do not propose alternatives. Where a "or" appears below, the recommendation is the first item; pick it unless there is a concrete blocker.

Layer	Choice
Monorepo	Turborepo + <code>pnpm</code> workspaces
Package manager	pnpm ≥ 9
Runtime	Node 22 LTS (set <code>.nvmrc</code> and <code>engines</code>)
Backend	Vendure 3.x (NestJS / TypeORM / GraphQL)
DB	PostgreSQL 16
Cache + Queue	Redis 7 + BullMQ via <code>@vendure/job-queue-plugin</code>
Search	Meilisearch v1.x (self-hosted; integrate via Vendure plugin or custom indexer)
CMS	Sanity.io (editorial/journal/athletes/about)
Image CDN	Bunny.net (Storage Zone + Pull Zone over Vendure asset origin)
Email	Resend + <code>react-email</code> templates
Payments	Mollie (primary) + Stripe (secondary, global cards)
Shipping	Sendcloud
Tax	Vendure tax zones (EU VAT)
Storefront	Next.js 15 App Router, RSC, Server Actions
UI	Tailwind CSS v4 + shadcn/ui primitives + custom design system
Motion	Framer Motion + native CSS
GraphQL client	urql (RSC-friendly) + graphql-codegen
Validation	Zod + next-safe-action
i18n	next-intl (App Router)
Analytics	Plausible (EU) + PostHog (EU instance)
Errors	Sentry
Logging	Better Stack (Logtail)
Uptime	UptimeRobot
Cookie consent	Klaro!
Hosting (backend)	Hetzner Cloud CX22 (Falkenstein/Nuremberg)

Layer	Choice
Hosting (storefront)	Vercel (default) — Cloudflare Pages as documented alternative
Reverse proxy	Caddy 2 (auto Let's Encrypt)
Containers	Docker + Docker Compose
DNS / WAF	Cloudflare
CI/CD	GitHub Actions
Tests	Vitest (unit) + Testing Library (components) + Playwright (e2e + visual) + Lighthouse CI
Lint/format	ESLint 9 flat config + Prettier + Husky + lint-staged
Secrets	<code>.env</code> locally, Doppler in CI/prod (document the exact wiring)

3. Monorepo Structure

Create exactly this layout. Every directory below must exist by end of Phase 1.

```

hurc/
├── .github/
│   ├── workflows/
│   │   ├── ci.yml                # lint+typecheck+test on PR
│   │   ├── e2e.yml              # Playwright on PR (storefront affected)
│   │   ├── deploy-backend.yml   # SSH deploy to Hetzner on main
│   │   ├── deploy-storefront.yml # Vercel deploy on main
│   │   ├── lighthouse.yml       # perf budgets on PR
│   │   └── codegen-check.yml     # ensure graphql types are committed & up-to-
date
│   ├── CODEOWNERS
│   └── pull_request_template.md
├── apps/
│   ├── backend/                  # Vendure server
│   │   ├── src/
│   │   │   ├── vendure-config.ts
│   │   │   ├── plugins/
│   │   │   │   ├── hurc-meilisearch/    # custom search indexer
│   │   │   │   ├── hurc-gpsr/           # GPSR data fields on products
│   │   │   │   ├── hurc-gdpr/           # data export/delete endpoints
│   │   │   │   ├── hurc-newsletter/     # Resend bridge
│   │   │   │   └── hurc-sanity-bridge/  # webhook receiver from Sanity
│   │   │   ├── migrations/
│   │   │   ├── email-templates/
│   │   │   └── index.ts
│   │   ├── static/
│   │   ├── test/
│   │   ├── Dockerfile
│   │   ├── tsconfig.json
│   │   └── package.json
│   └── storefront/              # Next.js 15
│       ├── src/
│       │   ├── app/
│       │   │   ├── [locale]/
│       │   │   └── (marketing)/         # journal, athletes, about,
sustainability
│       │   │   ├── (shop)/
│       │   │   │   ├── shop/[...segments]/ # PLP w/ filters
│       │   │   │   ├── product/[handle]/   # PDP
│       │   │   │   ├── cart/               # full cart fallback page
│       │   │   │   └── checkout/           # multi-step
│       │   │   ├── account/
│       │   │   └── legal/                 # imprint, privacy, terms,
withdrawal
│       │   ├── layout.tsx
│       │   └── page.tsx                 # home, video hero

```

```

├── api/
│   ├── revalidate/
│   ├── sanity/preview/
│   └── health/
├── opengraph-image.tsx
├── sitemap.ts
├── robots.ts
└── globals.css
├── components/
├── commerce/      # cart-drawer, add-to-cart, plp-card, pdp-
gallery, ...
│   ├── marketing/  # video-hero, editorial-grid, athlete-card
│   ├── layout/     # header, footer, country-switcher, trust-strip
│   └── primitives/  # re-exports / wrappers around @hurc/ui
├── lib/
│   ├── vendure/     # urql client, fragments, mutations
│   ├── sanity/      # client, queries, types
│   ├── actions/     # next-safe-action server actions
│   ├── analytics/
│   ├── intl/
│   └── seo/
├── middleware.ts    # locale + country detection
├── env.ts           # zod-validated env
├── messages/        # next-intl json files
├── public/
├── tests/
│   ├── e2e/         # playwright
│   └── unit/
├── playwright.config.ts
├── next.config.ts
├── tsconfig.json
└── package.json
├── packages/
│   ├── ui/          # design system (shadcn primitives + HURC
components)
│   ├── src/
│   │   ├── primitives/  # button, input, dialog, sheet, accordion, ...
│   │   ├── tokens/     # design tokens (colors, spacing, motion)
│   │   ├── icons/
│   │   └── index.ts
│   └── package.json
├── graphql/         # generated Vendure types + operations
│   ├── src/
│   │   ├── generated.ts  # codegen output (committed)
│   │   ├── fragments.graphql
│   │   ├── queries.graphql
│   │   └── mutations.graphql

```

```

├── codegen.ts
├── package.json
├── config-eslint/
├── config-tsconfig/      # base / nextjs / node / react-library presets
├── config-tailwind/      # shared preset + tokens
├── utils/                # money, locale, cookies, slug, etc.
├── analytics/            # Plausible + PostHog wrappers
├── email/                # react-email templates + Resend client
├── testing/              # shared test utils, MSW handlers, fixtures
├── infra/
│   ├── docker/
│   │   ├── docker-compose.yml      # local dev
│   │   └── docker-compose.prod.yml # Hetzner production
│   ├── caddy/
│   │   └── Caddyfile
│   ├── scripts/
│   │   ├── backup-postgres.sh      # pg_dump → R2
│   │   ├── restore-postgres.sh
│   │   ├── deploy.sh               # called by GH Actions
│   │   └── bootstrap-hetzner.sh     # one-shot server provisioning
│   └── README.md
├── docs/
│   ├── ARCHITECTURE.md
│   ├── RUNBOOK.md
│   ├── CONTRIBUTING.md
│   ├── SECURITY.md
│   ├── decisions/                  # ADRs
│   └── backlog.md
├── .editorconfig
├── .gitignore
├── .nvmrc
├── .prettierrc
├── eslint.config.js
├── package.json
├── pnpm-workspace.yaml
├── turbo.json
├── README.md
└── LICENSE

```

4. Phase-by-Phase Build Plan

Execute phases **in order**. After each phase: run all quality gates, write/update docs in `/docs`, commit, tag `phase-N-complete`.

1. Initialize git, `pnpm init`, set `"packageManager"` in root, write `.nvmrc` (22), `pnpm-workspace.yaml` covering `apps/*` and `packages/*`.
2. Add `turbo.json` with pipelines: `build`, `dev`, `lint`, `typecheck`, `test`, `test:e2e`, `codegen`. Use proper `dependsOn` and `outputs` (incl. `.next/**`, `dist/**`).
3. Create shared config packages: `config-tsconfig` (base / nextjs / node / react-library), `config-eslint` (flat config, TS + react + ally + import-x + tailwindcss + unicorn essentials), `config-tailwind` (Tailwind v4 preset exporting design tokens).
4. Root tooling: ESLint 9 flat config, Prettier with `prettier-plugin-tailwindcss`, Husky pre-commit running `lint-staged` (`eslint --fix`, `prettier --write`), commitlint with conventional config.
5. Create empty stubs of every package in §3 with minimal `package.json` + `tsconfig.json` that extends the shared preset. They must `pnpm typecheck` cleanly.
6. Write `/README.md` (overview + quickstart) and `/docs/CONTRIBUTING.md`.

Gate: `pnpm install && pnpm typecheck && pnpm lint` is clean.

Phase 2 — Vendure backend (`apps/backend`)

1. Scaffold with `npx @vendure/create` then port the structure into `apps/backend`. Pin to the latest stable `@vendure/core` 3.x — verify version with `pnpm view @vendure/core version`.
2. Configure `vendure-config.ts`:
 - `DefaultJobQueuePlugin` replaced with `BullMQJobQueuePlugin` pointing at Redis.
 - `AssetServerPlugin` with S3-compatible storage strategy pointing to **Bunny.net Storage Zone** (it speaks S3 API). Public URLs served via Bunny Pull Zone (configured as `assetUrlPrefix`).
 - `DefaultSearchPlugin` removed; install/build the **HURC Meilisearch plugin** (see Phase 2.6).
 - `EmailPlugin` wired to Resend (custom `EmailSender` implementation; templates compiled from `react-email`).
 - `AdminUiPlugin` with HURC-branded compiled admin (logo, colors, favicon).
 - `MolliePlugin` from `@vendure/payments-plugin` configured for EUR + GBP.
 - `StripePlugin` as secondary handler.
 - Channels: `default`, `eu`, `uk`. Tax zones for each EU country with correct VAT rates. Default currency EUR; `uk` channel GBP.
 - `cookieOptions`: `secure: true`, `sameSite: 'lax'`, signed.

3. Database: Postgres connection from env. Provide `(pnpm migration:generate)`/`(migration:run)` scripts. Commit an initial migration.
4. Custom fields on `(Product)` and `(ProductVariant)` for HURC:
 - `(activity)`: enum `(RUN | TRAIN | LIFT | REST)` (multi-select).
 - `(materialComposition)`: localizable string.
 - `(careInstructions)`: localizable rich-text.
 - `(sustainabilityNotes)`: localizable rich-text.
 - **GPSR block** (in `(hurc-gpsr)` plugin): `(responsiblePerson)` (struct: name, address, email), `(manufacturerInfo)`, `(warnings)`, `(traceabilityCode)`. Mandatory at publish time — admin UI validation.
5. Custom fields on `(Customer)`: `(marketingOptIn)`, `(marketingOptInAt)`, `(preferredActivity)`.
6. **`(hurc-meilisearch)` plugin**: subscribe to `(ProductEvent)` + `(ProductVariantEvent)` + `(StockMovementEvent)`; index documents with `{ id, sku, name, slug, description, price, currency, activity[], collections[], facets, inStock }`. Expose admin UI button "Reindex all". Use BullMQ for batch reindex.
7. **`(hurc-gdpr)` plugin**: authenticated REST endpoints `(POST /gdpr/export)` and `(POST /gdpr/delete)` (returns ZIP / soft-deletes PII while preserving order audit records per legal retention).
8. **`(hurc-newsletter)` plugin**: double-opt-in flow, integrates with Resend audiences.
9. **`(hurc-sanity-bridge)` plugin**: webhook endpoint receiving Sanity content updates → triggers Next.js revalidation via signed `(revalidate)` call.
10. Seed script (`(pnpm seed)`) creating: 2 collections per activity (12 total), 8 demo products with variants (sizes XS-XXL, 2 colors), countries, zones, tax rates, shipping methods (Sendcloud-backed), payment methods (Mollie + Stripe), 1 admin user, 1 test customer.

Gate: `(pnpm --filter backend dev)` runs; admin UI loads at `(/admin)`; GraphQL playground at `(/shop-api)`; seed completes; reindex job succeeds; integration tests (Vitest) for each plugin pass.

Phase 3 — GraphQL layer (`(packages/graphql)`)

1. Add `(graphql-codegen)` with `(client-preset)` (or `(typescript-operations)` + `(typed-document-node)`) targeting `(http://localhost:3000/shop-api)`.
2. Author fragments: `(CartFragment)`, `(OrderFragment)`, `(ProductSummaryFragment)`, `(ProductDetailFragment)`, `(CollectionFragment)`, `(CustomerFragment)`, `(AddressFragment)`.
3. Author all queries/mutations needed by storefront: `(activeOrder)`, `(addItemToOrder)`, `(adjustOrderLine)`, `(removeOrderLine)`, `(applyCouponCode)`, `(setOrderShippingAddress)`, `(setOrderShippingMethod)`, `(transitionOrderToState)`, `(addPaymentToOrder)`, `(nextOrderStates)`, `(eligibleShippingMethods)`, `(eligiblePaymentMethods)`,

`product(slug)`, `products(filter)` (+ Meilisearch facet results), `collection(slug)`, `me`, `login`, `logout`, `register`, `requestPasswordReset`, `resetPassword`, `updateCustomer`, etc.

4. Codegen output committed at `packages/graphql/src/generated.ts`. CI must fail if codegen drifts (workflow `codegen-check.yml`).

Gate: `pnpm --filter @hurc/graphql codegen && pnpm typecheck` clean.

Phase 4 — Storefront foundation (`apps/storefront`)

1. Scaffold Next.js 15 with App Router, TypeScript, Tailwind. Verify `next` version with `pnpm view next version`. Enable `experimental: { ppr: 'incremental' }` if stable; otherwise leave off and document.
2. `src/env.ts`: Zod-validated env with `serverEnv` and `clientEnv` separation. Throw at startup if missing.
3. **Vendure client (`lib/vendure`):**
 - urql client factory with two flavors: `getServerClient(cookies)` (RSC, forwards `vendure-auth-token` + `vendure-session` cookies) and `getClientClient()` (browser).
 - All Server Actions read/set cookies from `next/headers`.
 - Active-order pattern: a `getActiveOrder()` helper memoized per request.
4. **i18n with next-intl**: locales `en, de, fr, nl, es, it`. `[locale]` segment. Middleware detects via Cloudflare `cf-ipcountry` header → maps country → channel + locale. Country selector (modal) lets users override; selection stored in cookie.
5. **Country/currency switching**: switching country swaps Vendure channel via `vendure-token` header on subsequent requests; storefront re-fetches the active order in the new channel context.
6. **Theming**: Tailwind v4 with CSS variables — define semantic tokens (`--color-bg`, `--color-fg`, `--color-accent`, `--color-muted`, `--color-line`) for `:root` (dark default) and a `[data-theme="light"]` override. Brand palette per §1.
7. **Fonts**: `next/font/local` (or `next/font/google` with subset `latin, latin-ext`) for display + body. `font-display: swap`.
8. **SEO foundations**: root `layout.tsx` exporting default `metadata`; `generateMetadata` for product/collection/page; `sitemap.ts` querying Vendure for products/collections + Sanity for editorial; `robots.ts`; `opengraph-image.tsx` using `ImageResponse`; JSON-LD helper in `lib/seo` for `Organization`, `WebSite`, `BreadcrumbList`, `Product`, `Article`.
9. **Analytics**: Plausible script (no cookies) + PostHog (loaded only after Klaro consent). Custom events: `view_item`, `add_to_cart`, `begin_checkout`, `purchase`,

`newsletter_signup`, `country_change`.

10. **Error tracking:** Sentry for both server and client; source maps uploaded in CI; PII scrubbing on.
11. **Logging:** server-side pino piped to Better Stack via HTTP transport; redact `authorization`, `cookie`, `email` by default.

Gate: `pnpm --filter storefront dev` boots; locale routing works; Vendure client returns active order; Lighthouse on the empty home page ≥ 95 across the board.

Phase 5 — Design system (`packages/ui`)

1. Set up shadcn/ui with the **monorepo pattern**: components live in `packages/ui/src/primitives` and are imported by storefront via `@hurec/ui`. Configure `components.json` accordingly.
2. Primitives: `Button`, `Input`, `Textarea`, `Select`, `Checkbox`, `RadioGroup`, `Switch`, `Dialog`, `Sheet` (drawer), `Drawer`, `Accordion`, `Tabs`, `Tooltip`, `Toast` (sonner), `DropdownMenu`, `Command`, `ScrollArea`, `Separator`, `Skeleton`, `Avatar`, `Badge`, `Progress`. All wrapped to consume HURC tokens.
3. Composite components: `PriceTag`, `ColorSwatch`, `SizePicker`, `QuantityStepper`, `TrustStrip`, `FreeShippingProgress`, `KlarnaInstallmentLine`, `CountryCurrencyModal`, `NewsletterForm`, `ReviewStars`, `BreadcrumbTrail`.
4. Motion: `Reveal` (intersection-driven fade-up), `Marquee`, `HoverImageSwap`, `StickyColumn`. Respect `prefers-reduced-motion`.
5. **Accessibility checklist** baked into Storybook (or `tests/a11y` if no Storybook):
 - Focus visible always (custom ring using accent color at 30% opacity).
 - Focus trap + return-focus for `Dialog`, `Sheet`, `Drawer`.
 - `aria-live="polite"` region for cart updates and toasts.
 - All icon-only buttons have `aria-label`.
 - Color contrast $\geq 4.5:1$ for body text (verify against the dark theme).
6. Visual regression: Playwright snapshots per primitive in `packages/ui/tests`.

Gate: Component tests pass; axe scan clean; visual snapshots committed.

Phase 6 — Commerce flows (`apps/storefront`)

Implement, in this order, with full UI + Server Actions + tests:

1. **Header + Footer + Trust Strip**
 - Sticky transparent $\rightarrow$ solid header on scroll.

- Activity-first nav: `(RUN / TRAIN / LIFT / REST)` plus `(JOURNAL / ATHLETES)`. Mega-menu on desktop, full-screen drawer on mobile.
- Cart icon with item count (live region).
- Country/currency switcher modal.
- Footer with newsletter, social, legal links, payment marks.

2. Home page (`(app/[locale]/page.tsx)`)

- Cinematic muted-autoplay video hero (preload `(metadata)`, poster image, fallback `(<img>)`). Playback controls accessible. Use Bunny Stream or self-hosted MP4/WebM.
- Activity tiles (4 cards linking to PLPs).
- Editorial blocks pulled from Sanity.
- Featured products carousel.
- Newsletter capture.

3. PLP (`(app/[locale]/(shop)/shop/[...segments]/page.tsx)`)

- Server-rendered with streaming Suspense for facets.
- Filter chips (activity, size, color, price range, in-stock). URL-driven state (`(?size=M&color=black)`).
- Sort dropdown.
- Product card: dual-image hover swap, color swatches that swap the displayed image, quick-add button (opens size sheet on mobile, hover popover on desktop).
- Infinite scroll OR paginated — pick paginated for SEO; document the choice in an ADR.

4. PDP (`(app/[locale]/(shop)/product/[handle]/page.tsx)`)

- Sticky gallery (left) + sticky purchase column (right) on desktop $\geq$ `(lg)`.
- Mobile: gallery on top, sticky bottom CTA bar.
- Variant selection: color $\rightarrow$ size, with availability + low-stock badge.
- Klarna installment line: "or 3 payments of €X" computed from variant price (only show in countries where Klarna is enabled by Mollie).
- Accordion: Description, Materials & Care, Size & Fit, Shipping & Returns, **GPSR block** (responsible person, manufacturer, warnings, traceability code).
- Complete-the-look strip (related products via Vendure facet filter or Sanity-curated list).
- Reviews section (placeholder data model + UI; integration with a provider like Trustpilot/Junip can be a later phase but the UI ships).
- JSON-LD `(Product)` + `(BreadcrumbList)`.

5. Cart drawer

- Slide-in `Sheet` from the right.
- Free-shipping progress bar (threshold per channel from Vendure config).
- Line items with thumbnail, variant, quantity stepper, remove.
- Subtotal, estimated tax, "Checkout" CTA.
- Empty state.
- Optimistic UI on quantity changes via Server Actions + `useOptimistic`.

6. Checkout (`app/[locale]/(shop)/checkout/page.tsx`)

- Single-page, three-step (Contact/Address → Shipping → Payment) with progress indicator.
- Guest checkout default.
- Address autocomplete (optional; if used, integrate Loqate/Google — otherwise plain inputs with VAT-ID field for B2B).
- Shipping method radio cards (Sendcloud-backed via Vendure).
- Payment: Mollie components (iDEAL, Bancontact, Klarna, SEPA, Cards, Apple/Google Pay) + Stripe Elements as fallback.
- Order confirmation page reads `?orderCode=` and queries Vendure.

7. Account (`app/[locale]/account/...`)

- Login / Register / Forgot / Reset.
- Dashboard, Orders list, Order detail (with tracking link from Sendcloud), Addresses, Profile, Marketing preferences, **GDPR data export & delete** buttons.

Every Server Action uses `next-safe-action` with Zod input schema + typed errors. No raw `fetch` to Vendure from client components.

Gate: Playwright e2e suite covers: browse → add to cart → checkout → pay (Mollie test mode) → account view. All pass on CI.

Phase 7 — Editorial / Sanity

1. Sanity studio embedded at `apps/storefront/src/app/studio/[...index]/page.tsx` (mounted under `/studio`), gated by Sanity auth).
2. Schemas: `journalPost` (title, slug, hero, body Portable Text, author, tags, publishedAt, seo), `athlete` (name, sport, bio, hero video, stats, gallery), `aboutPage`, `sustainabilityPage`, `homepageBlocks`.
3. Pages:
 - `/journal` index + `/journal/[slug]` detail.
 - `/athletes` index + `/athletes/[slug]` detail.
 - `/about`, `/sustainability`.

4. Draft mode: `/api/sanity/preview` enables draft cookie; `disable-draft` route to clear. Sanity webhook hits `/api/revalidate` with HMAC signature.
5. Portable Text components themed to HURC (pull-quote, full-bleed image, video, product-embed referencing Vendure slugs).

Gate: Editor can publish a post in draft, preview it, hit publish, see it live within seconds via on-demand revalidation.

Phase 8 — EU compliance & legal

1. **Klaro!** consent banner configured for: Plausible (always on, no consent needed since cookieless), PostHog (consent required), Sentry (consent required for replay; errors are legitimate interest), marketing pixels (consent required).
2. Legal pages with templated content (English + German at minimum, others scaffolded):
 - `/legal/imprint` (Impressum) — static template with placeholders the user fills.
 - `/legal/privacy` — GDPR-compliant template referencing PostHog/Sentry/Plausible/Vendure.
 - `/legal/terms`.
 - `/legal/withdrawal` — 14-day right of withdrawal + downloadable model form (PDF generated at build time or as a static asset).
 - `/legal/shipping-returns`.
 - `/legal/cookies` — auto-rendered from Klaro config.
3. **VAT-inclusive pricing** on all customer-facing surfaces; `incl. VAT` label localized.
4. **GPSR:** PDP mandatory data block (already in Phase 6). Admin validation prevents publishing a product without it (already in Phase 2).
5. **GDPR endpoints** wired in account UI calling backend `hurc-gdpr` plugin. Confirmation email on delete.
6. Cookie-less analytics by default; consent gating implemented and tested (Playwright test that no PostHog cookie is set without consent).

Gate: `npx @axe-core/cli` against all key pages clean; manual review of legal pages by user (this step is approved automatically — ship templates).

Phase 9 — Testing & quality

1. **Vitest** unit tests for: utility functions, money/locale helpers, Zod schemas, Vendure plugins (with `@vendure/testing`).
2. **Testing Library** component tests for every primitive in `@hurc/ui`.
3. **Playwright** e2e suites:
 - `commerce.spec.ts` — full purchase journey.

- `account.spec.ts` — register, login, change password, GDPR export.
 - `i18n.spec.ts` — locale switch + currency change.
 - `seo.spec.ts` — JSON-LD presence, hreflang, canonical, sitemap accessibility.
 - `a11y.spec.ts` — `@axe-core/playwright` on top 10 pages.
4. **Visual regression** snapshots for key surfaces (home, PLP, PDP, cart drawer, checkout step 1).
 5. **GraphQL contract tests**: a Vitest suite that runs codegen against the live backend in CI and asserts the committed schema matches.
 6. **Lighthouse CI** with budgets: LCP $\leq$ 2.0s, INP $\leq$ 200ms, CLS $\leq$ 0.05, JS $\leq$ 180KB transfer on home, $\leq$ 220KB on PDP. Fails PR if regressed.
 7. Coverage gate: 80% lines for `packages/utils`, `packages/ui` primitives, and Server Actions.

Gate: `pnpm test && pnpm test:e2e && pnpm lighthouse` all green.

Phase 10 — Infra, CI/CD, deployment

1. **Local dev compose** (`infra/docker/docker-compose.yml`): Postgres 16, Redis 7, Meilisearch v1, MailHog. Volumes for data. Single `pnpm dev:up`.
2. **Production compose** (`infra/docker/docker-compose.prod.yml`):
 - `vendure-server` (built from `apps/backend/Dockerfile`, multi-stage).
 - `vendure-worker` (same image, different command).
 - `postgres` with named volume + healthcheck.
 - `redis` with append-only persistence.
 - `meilisearch` with master key from env.
 - `caddy` reverse-proxying `api.hurc.com` $\rightarrow$ server, `admin.hurc.com` $\rightarrow$ admin, `assets.hurc.com` $\rightarrow$ asset server. Auto-TLS.
 - All on a private network; only Caddy exposes 80/443.
3. **Bootstrap script** (`infra/scripts/bootstrap-hetzner.sh`): provisions a fresh CX22 — system updates, ufw, fail2ban, docker, non-root deploy user with SSH key, `/srv/hurc` directory, swap file (since CX22 is 4 GB RAM), unattended-upgrades, log rotation.
4. **Backups**: `backup-postgres.sh` runs nightly via cron, `pg_dump | gzip | rclone copyto bunny-r2:/hurc/backups/` (use Cloudflare R2 — credentials in env). 30-day retention. Restore script tested in the runbook.
5. **GitHub Actions**:
 - `ci.yml`: install (cache pnpm), `turbo run lint typecheck test build` with affected detection, upload coverage.

- `e2e.yml`: spin up backend in services, run Playwright, upload traces on failure.
 - `lighthouse.yml`: run against preview URL.
 - `deploy-backend.yml`: on `main`, build + push Docker image to GHCR, SSH to Hetzner via `appleboy/ssh-action`, pull image, `docker compose up -d --no-deps --build vendure-server vendure-worker`, run migrations, healthcheck, rollback on failure.
 - `deploy-storefront.yml`: triggers Vercel deploy via the Vercel GitHub integration; we keep this workflow for manual triggers + status check aggregation.
 - `codegen-check.yml`: regenerates `packages/graphql/src/generated.ts` and fails if `git diff` is non-empty.
6. **Secrets:** Doppler project `hurc` with configs `dev`, `stg`, `prd`. CI pulls via `dopplerhq/cli-action`. Local devs use `doppler run -- pnpm dev` (documented in `CONTRIBUTING.md`).
7. **Cloudflare:** DNS records, proxy on for `www`/`api`/`admin`/`assets`, WAF managed rules + custom rule blocking `/admin*` from outside trusted IPs (configurable), caching rules bypassing `/shop-api`, `/admin-api`, `/checkout`.
8. **Monitoring:**
- Sentry projects: `hurc-storefront`, `hurc-backend`.
 - UptimeRobot monitors: storefront home, `api/health`, `admin/health`, `sitemap.xml`. 1-min cadence.
 - Better Stack log sources from Caddy, Vendure server, Vendure worker, Next.js.

Gate: Push to `main` deploys backend to Hetzner and storefront to Vercel; healthchecks green; first synthetic order via Mollie test mode succeeds end-to-end.

5. Code style & quality standards

- **TypeScript:** strict, no implicit any, no non-null assertions in production code (only in tests with comment), prefer `type` over `interface` for object shapes unless extending. Discriminated unions for results.
- **React:** Server Components by default; reach for `"use client"` only when interactivity demands it. Co-locate Server Actions in `lib/actions/<domain>.ts`. Use `useOptimistic` and `useFormStatus` where appropriate.
- **Naming:** `PascalCase` for components, `camelCase` for functions/vars, `kebab-case` for files except components (`ProductCard.tsx`). One component per file. Folders are `kebab-case`.
- **Imports:** absolute via TS paths (`@/components/...`, `@hurc/ui`, `@hurc/graphql`). Sorted by `eslint-plugin-import-x` with `simple-import-sort`.

- **CSS:** Tailwind utility-first; class merging with `cn()` (clsx + tailwind-merge); semantic tokens before raw colors; arbitrary values discouraged — extend the preset.
 - **GraphQL:** all operations live in `packages/graphql`, never inlined ad-hoc in components.
 - **Errors:** all Server Actions return `Result<T>`; throw only for programmer errors. UI converts errors to localized toasts via `messages/*.json` keys.
 - **Money:** always store as integer minor units; format via `Intl.NumberFormat` with channel currency.
 - **Comments:** explain *why*, not *what*. JSDoc on exported APIs of `packages/*`.
-

6. Environment variables

Maintain `apps/*/env.example` and a single `infra/env.reference.md` listing every variable, its scope (server/client), and where it's set (Doppler config). At minimum:

```
# — Backend —————
DATABASE_URL=
REDIS_URL=
COOKIE_SECRET=
SUPERADMIN_USERNAME=
SUPERADMIN_PASSWORD=
ASSET_BUNNY_S3_ACCESS_KEY=
ASSET_BUNNY_S3_SECRET_KEY=
ASSET_BUNNY_S3_BUCKET=
ASSET_BUNNY_S3_ENDPOINT=
ASSET_PUBLIC_URL_PREFIX=
MEILI_HOST=
MEILI_MASTER_KEY=
MOLLIE_API_KEY=
STRIPE_SECRET_KEY=
STRIPE_WEBHOOK_SECRET=
SENDCLLOUD_PUBLIC_KEY=
SENDCLLOUD_SECRET_KEY=
RESEND_API_KEY=
SENTRY_DSN_BACKEND=
LOGTAIL_SOURCE_TOKEN_BACKEND=
SANITY_WEBHOOK_SECRET=
NEXT_REVALIDATE_SECRET=

# — Storefront —————
NEXT_PUBLIC_VENDURE_SHOP_API_URL=
VENDURE_SHOP_API_URL_INTERNAL=
NEXT_PUBLIC_SITE_URL=
NEXT_PUBLIC_PLAUSIBLE_DOMAIN=
NEXT_PUBLIC_POSTHOG_KEY=
NEXT_PUBLIC_POSTHOG_HOST=
NEXT_PUBLIC_SANITY_PROJECT_ID=
NEXT_PUBLIC_SANITY_DATASET=
SANITY_API_READ_TOKEN=
SANITY_REVALIDATE_SECRET=
SENTRY_DSN_STOREFRONT=
LOGTAIL_SOURCE_TOKEN_STOREFRONT=
MOLLIE_PROFILE_ID=
STRIPE_PUBLISHABLE_KEY=
```

`src/env.ts` validates with Zod at boot; missing vars crash with a readable message.

7. Documentation deliverables (must exist by Phase 10)

- `README.md` — what HURC is, quickstart (`pnpm i && pnpm dev`), links to docs.
 - `docs/ARCHITECTURE.md` — system diagram (mermaid), data flow, request flow for "view PDP" and "checkout", channel/locale model, asset pipeline.
 - `docs/RUNBOOK.md` — how to deploy, roll back, restore DB, rotate secrets, reindex Meilisearch, replay BullMQ failed jobs, handle Mollie webhook outages.
 - `docs/CONTRIBUTING.md` — local setup, branching, commit convention, PR checklist, codegen workflow.
 - `docs/SECURITY.md` — threat model summary, secret handling, dependency policy (Renovate/Dependabot configured), responsible disclosure address.
 - `docs/decisions/` — at minimum ADRs for: monorepo tool choice, GraphQL client, CMS choice (Sanity vs Payload), pagination vs infinite scroll on PLP, hosting split (Hetzner backend / Vercel storefront).
 - `docs/backlog.md` — anything explicitly deferred.
-

8. Acceptance criteria — definition of "done"

The build is shippable when **all** of the following hold:

- ☐ `pnpm install && pnpm typecheck && pnpm lint && pnpm test && pnpm test:e2e && pnpm build` are green from a clean checkout.
- ☐ `docker compose -f infra/docker/docker-compose.yml up` boots the full local stack; storefront serves on `:3001`, backend on `:3000`, admin loads, GraphQL responds, seed data is browsable.
- ☐ Lighthouse CI passes the budgets in §4 Phase 9 on home, PLP, PDP.
- ☐ A test order placed in Mollie test mode completes through to "Payment Settled" in Vendure admin and triggers an order confirmation email rendered by Resend.
- ☐ Country switcher changes channel + currency; prices and shipping methods update accordingly.
- ☐ All six locales render the home page without missing-translation warnings.
- ☐ Klaro consent banner blocks PostHog/Sentry-replay until accepted; Plausible loads regardless.
- ☐ PDP shows GPSR block; admin cannot publish a product missing GPSR data.
- ☐ GDPR export downloads a ZIP of the test customer's data; GDPR delete removes PII while keeping order audit rows.
- ☐ `sitemap.xml` lists all products, collections, journal posts, athletes, and static pages, with correct hreflang alternates.
- ☐ `axe` clean on home, PLP, PDP, cart drawer, checkout, account dashboard.

- ☐ Sentry receives test errors from both apps; UptimeRobot monitors are green; Better Stack ingests logs.
 - ☐ One push to `main` deploys both apps; rollback procedure documented and rehearsed once.
-

9. Decision principles (use when this spec is silent)

1. **Server-first.** Default to RSC/Server Actions. Reach for client only with a written reason in a comment.
 2. **Performance is a feature.** If a choice trades 50ms of LCP for nicer DX, refuse it.
 3. **Boring tech wins.** Prefer the documented, popular path.
 4. **Localize early.** Never hard-code user-facing strings; always go through `next-intl`.
 5. **Type safety is end-to-end.** From DB column to React prop, types should flow without `any`.
 6. **Fail loudly in dev, gracefully in prod.** Use error boundaries and friendly fallbacks; log everything to Sentry/Better Stack.
 7. **Security defaults on.** New surfaces ship with auth, rate limit, CSRF protection, CSP, validated inputs.
-

10. Kickoff checklist (run before Phase 1)

1. Confirm Node 22 LTS is installed; `corepack enable && corepack prepare pnpm@latest -> activate`.
 2. `git init && git commit --allow-empty -m "chore: initialize hurc monorepo"`.
 3. Create `.gitignore` covering `node_modules`, `.next`, `dist`, `.turbo`, `.env*`, `playwright-report`, `coverage`, `*.log`.
 4. Create `docs/decisions/0001-stack.md` recording this prompt verbatim as the source of truth.
 5. Begin **Phase 1**.
-

Now build it. Phase 1 first. Stop after each phase, run the gates, commit, tag, then continue. Do not output a summary at the end — output the working code, the working CI, and the deployed services.